

No Evidence That TaperNorm Increases Forgetting After a Web-to-Code Domain Shift: A Pre-Specified Pilot Study

[Author: Author name]
[Author: Affiliation, or “Independent researcher”]
[Author: email]

First draft — not for distribution

Abstract

TaperNorm (Kanavalau et al., 2026) replaces the internal normalization layers of a pre-norm Transformer with a gated map that behaves like RMSNorm early in training and then tapers to a fixed, calibrated, sample-independent scaling. A tapered layer no longer rescales each token by its own statistics, which raises a natural concern: such a model might forget more when its training distribution shifts. We tested this concern in a small pilot whose design, primary endpoint and decision rules were specified before training. Small decoder-only language models (17.7M parameters), using either internal RMSNorm or internal TaperNorm, were trained on 150M tokens of web text (C4). Each was then continued for 100M tokens either on further web text or on Python code (The Stack), with three paired seeds. The primary endpoint is a difference-in-differences D of held-out web cross-entropy, where positive values mean more excess forgetting under TaperNorm. Switching to Python raised web cross-entropy by about 1.66 nats/token in both conditions. Mean D was -0.0223 nats/token (seeds -0.0200 , -0.0207 , -0.0262), which is below the pre-specified $+0.015$ bound for a meaningful excess. The pre-specified decision is therefore to stop: we find no evidence that TaperNorm increases persistent forgetting in this setting. The endpoint difference points the opposite way in all three seeds, but it is small, it falls short of the pre-specified threshold for an opposite-direction effect, and the seed-mean contrast varied between -0.024 and $+0.024$ across intermediate checkpoints, so we do not claim that TaperNorm reduces forgetting. Code, the frozen configuration and all evaluation records are released.

1 Introduction

Normalization layers such as LayerNorm (Ba et al., 2016) and RMSNorm (Zhang and Sennrich, 2019) are a standard part of Transformer language models. In the common pre-norm arrangement (Xiong et al., 2020), every attention and MLP sub-layer reads its input through a normalizer that rescales each token’s activation vector by that token’s own statistics. A long line of work has asked whether normalization is necessary at all (Zhang et al., 2019; Brock et al., 2021). TaperNorm (Kanavalau et al., 2026) is a recent proposal for Transformers. It acts exactly as RMSNorm (or LayerNorm) during an initial gate warm-up, calibrates a fixed scaling branch during that warm-up, and then cosine-decays a global gate until each internal normalizer becomes a sample-independent linear map. That map can be folded into the adjacent linear projections at inference. Kanavalau et al. (2026) report that this matches normalized baselines under identical setups, and they measure up to $1.22\times$ higher throughput in an inference microbenchmark.

This study looks at a property that the original evaluation did not target: behaviour under a later shift in the training distribution. A normalizer is invariant to the scale of its input. If continued training on a new domain changes activation magnitudes, an RMSNorm layer absorbs part of that change automatically, whereas a tapered layer applies a scale calibrated on the earlier distribution. One can conjecture that the tapered network must then absorb the shift through weight updates, which might overwrite structure that supports the original domain and so increase catastrophic forgetting (McCloskey and Cohen, 1989; French, 1999; Kirkpatrick et al., 2017). We use this argument only to motivate the hypothesis; we do not test the mechanism.

Hypothesis. H1: after training switches from web text to Python code, internal TaperNorm (without the auxiliary scale loss; final RMSNorm retained) increases the persistent deterioration of held-out web cross-entropy, relative to continuing web training, compared with internal RMSNorm.

We ran a deliberately small, inexpensive pilot to decide whether H1 deserves a larger study. Its design (Section 3) pairs the two conditions on initialization and data order and branches each condition’s trained prefix into a web continuation and a Python continuation from an identical saved state. The design also fixes a difference-in-differences endpoint and an ordered set of decision rules before any training. Our contributions are:

- a paired, pre-specified design for measuring how an architectural choice affects forgetting under a domain shift, whose primary contrast decomposes exactly into branch-level forgetting, a control-branch term, per-document effects and per-token-class effects;
- the pilot’s result. All validity checks pass and the pre-specified decision is *stop: small observed effect*. Mean $D = -0.0223$ nats/token, against a $+0.015$ bound for a meaningful excess;
- code, frozen configuration, data hashes and all evaluation records, sufficient to recompute every number in this paper.

The study is small (17,718,784 parameters, three seeds, one domain pair) and should be read as a screening result, not as a general statement about TaperNorm (Section 7).

2 Related work

Normalization and its removal. LayerNorm (Ba et al., 2016) and RMSNorm (Zhang and Sennrich, 2019) normalize each token independently, and the pre-norm placement (Xiong et al., 2020) is now standard in decoder-only language models. Fixup (Zhang et al., 2019) and NFNet (Brock et al., 2021) train deep residual networks without normalization through initialization, scaling and clipping choices. TaperNorm (Kanavalau et al., 2026) instead removes normalization gradually during training. Its analysis attributes the essential role of normalization to the *output* normalizer, which anchors scale; this is why our Taper-minus condition keeps the final RMSNorm.

Forgetting under continued training. Catastrophic forgetting (McCloskey and Cohen, 1989; French, 1999) is the loss of earlier competence when a network is trained on new data, and it has motivated methods such as elastic weight consolidation (Kirkpatrick et al., 2017). In language modelling, continued pre-training on a new domain improves in-domain performance (Gururangan et al., 2020), but it can degrade performance on the original distribution. Ibrahim et al. (2024) study how learning-rate re-warming, re-decaying and replay trade adaptation to the new distribution against this forgetting. Our design deliberately uses no replay, re-warming or other mitigation. It isolates how much forgetting the normalization choice itself causes, relative to a matched control branch.

Pre-specification. Fixing hypotheses, endpoints and analysis rules before seeing outcomes protects against flexible analysis (Nosek et al., 2018). It is still uncommon in machine-learning experiments. Section 3.6 states exactly what was fixed, and when.

3 Pre-specified design

3.1 Estimands

Let $L(m, b, s)$ be the token-weighted cross-entropy on the held-out web development set (nats per supervised token, 2,097,152 labels), for normalization condition $m \in \{\text{R}, \text{T}\}$ (RMS, Taper-minus), after s continuation updates on branch $b \in \{\text{web}, \text{code}\}$. Let $L(m, \text{prefix})$ be the same quantity at the switch. The primary endpoint, evaluated at $s = 6,104$, is

$$D(s) = [L(\text{T}, \text{code}, s) - L(\text{T}, \text{web}, s)] - [L(\text{R}, \text{code}, s) - L(\text{R}, \text{web}, s)]. \quad (1)$$

Positive D means that, relative to its own web-continuation control, the Taper-minus model loses more web performance under the Python shift than the RMS model does. Within each condition the code-minus-web difference cancels everything the two branches share, including the condition-specific quality of the prefix and generic continued-training drift. We also report the actual forgetting $F(m, b) = L(m, b, 6104) - L(m, \text{prefix})$ and the change in the between-condition gap during web continuation, $G_{\text{web}} = [L(\text{T}, \text{prefix}) - L(\text{R}, \text{prefix})] - [L(\text{T}, \text{web}, 6104) - L(\text{R}, \text{web}, 6104)]$. These satisfy the identity

$$D = \underbrace{F(\text{T}, \text{code}) - F(\text{R}, \text{code})}_Q + G_{\text{web}}, \quad (2)$$

so $Q = D - G_{\text{web}}$ is the direct difference in Python-branch forgetting. The identity is checked numerically in the analysis code. Because CE is token-weighted, D decomposes exactly into per-document terms D_i (weighted by each document’s scored-label count) and into four disjoint GPT-2 token classes: whitespace (W), text containing an alphanumeric character (A), other punctuation and symbols (P), and special, control or invalid-fragment tokens (X).

3.2 Models and conditions

All models are pre-norm decoder-only Transformers adapted from nanoGPT (Karpathy, 2022):

- 6 layers, width 256, 4 heads of size 64, and an exact-GELU MLP of width 1,024;
- learned absolute positions, context 512, no biases and no dropout;
- GPT-2 token embeddings (Radford et al., 2019), tied to the output layer (vocabulary 50,257).

The RMS model has 17,718,784 parameters; Taper-minus adds 3,072 for its second gain vectors. About 73% of all parameters are in the token-embedding table. Both conditions keep the final RMSNorm.

RMS. All 12 internal normalizers are RMSNorm with $\epsilon = 10^{-6}$.

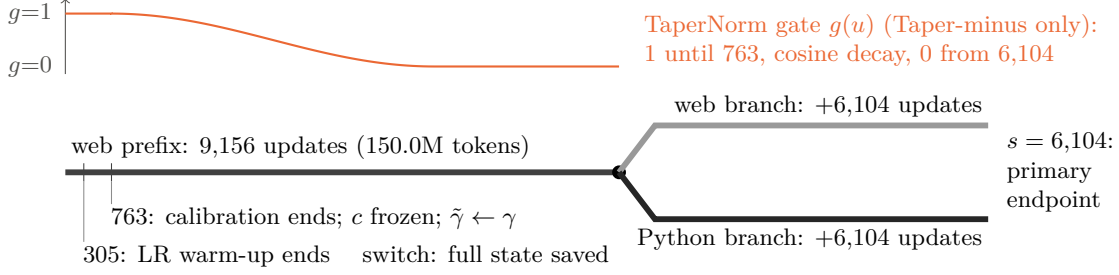


Figure 1: One condition and seed of the design. Both conditions start from identical initial weights and read the same data order for each seed. At the switch, each condition’s complete state (weights, optimizer moments, loss scale and random-number state) is saved, and both of its branches restore that identical state. The web branch continues on C4; the Python branch trains on Python files from The Stack. The global learning-rate clock continues through the branches without a restart. Evaluations follow the schedule in Appendix A.

Taper-minus. All 12 internal normalizers are TaperNorm, following the operator and calibration of Kanavalau et al. (2026) (their Section 3 and Appendix C), without their auxiliary scale loss. For an input $h \in \mathbb{R}^d$ at optimizer update u , with gains γ and $\tilde{\gamma}$,

$$r(h) = \sqrt{\|h\|_2^2/d + 10^{-6}}, \quad y = g(u) \frac{h}{r(h)} \odot \gamma + (1 - g(u)) c h \odot \tilde{\gamma}. \quad (3)$$

The scalar c of each site is calibrated during updates 1–763 as the ratio of bias-corrected exponential moving averages (one EMA step per optimizer update, new-sample weight 0.01) of $\|h \odot \gamma\|^2/r(h)$ and $\|h \odot \gamma\|^2$. These are collected from the pre-step forward passes of every microbatch. After update 763, c is frozen and $\tilde{\gamma}$ is initialized from γ ; $\tilde{\gamma}$ first receives an optimizer step at update 764 and stays trainable. The gate is $g(u) = 1$ for $u \leq 763$, $g(u) = \frac{1}{2}[1 + \cos(\pi(u - 763)/5341)]$ for $763 < u < 6104$, and $g(u) = 0$ for $u \geq 6104$. At $g = 0$ no per-token statistic is computed and γ receives no gradient. The gate timing, GELU, learned positions, weight decay, learning-rate schedule and domain branching are choices of this study, so this is an adaptation of TaperNorm, not a replication of its experiments.

3.3 Training protocol

Optimization. The optimizer is AdamW (Loshchilov and Hutter, 2019) with $\beta = (0.9, 0.95)$ and $\epsilon = 10^{-8}$. All two-dimensional parameters, embeddings included, get weight decay 0.1; gain vectors get none. Gradients are clipped once per update to global norm 1.0, after accumulation. Each update uses $32 \times 512 = 16,384$ supervised tokens. The learning rate rises linearly to 6×10^{-4} over 305 updates and then follows a cosine decay to 6×10^{-5} at global update 15,260. The branches continue this global clock, with no learning-rate restart, optimizer reset, replay or regularization toward the prefix (Figure 1).

Pairing. Seeds 101, 102 and 103 are paired across conditions. A canonical RMS initialization is generated from the seed and explicitly copied into the Taper-minus model; $\tilde{\gamma}$ starts at 1 and is inactive until update 764. Both conditions read the same pre-generated window order. Each run consists of a prefix of 9,156 updates (150,011,904 tokens) followed by two branches of 6,104 updates each (100,007,936 tokens). The web branch reads further unseen web windows and the Python branch reads Python windows; no window repeats within a trajectory. The primary allocation is 2.10B supervised tokens.

Precision. Training ran in FP16 autocast with dynamic loss scaling (Micikevicius et al., 2018). Master weights, optimizer moments, normalization and calibration statistics, and the loss were kept in FP32. A step whose gradients were nonfinite was to be retried on the same batch at a halved loss scale, so no update is ever skipped. This replaced the protocol’s original BF16 plan, because the available hardware (NVIDIA T4) lacks efficient BF16 support (Table 2).

3.4 Data

Sources. Web text comes from C4 English (Raffel et al., 2020) (revision 1588ec454efa); training uses the official training split, and the official validation split is hashed 50/50 into development and reserved-test sets. Code comes from the Python subset of The Stack (deduplicated) (Kocetkov et al., 2022) (revision 17cad72c886a). Files are grouped into connected components over their repository-name aliases (140,195 groups), and each group is assigned by $\text{sha256}(\text{group}) \bmod 100$: 0–89 train, 90–94 development, 95–99 test.

Deduplication. Documents are deduplicated exactly by content hash across all domains and splits. An approximate near-duplicate audit uses MinHash (Broder, 1997) over GPT-2 token 5-grams (128 permutations, LSH with 32 bands of 4 rows), confirmed by exact Jaccard similarity ≥ 0.85 . When duplicates straddle splits, the copy in the higher-priority split is kept (test over development over training). This removed 0 exact and 2,182 near duplicates.

Packing and quotas. Text is tokenized with GPT-2 BPE, with one EOS appended per document, and packed into 513-token windows at stride 512 with no padding. Quotas are 260,000,257 web and 110,000,129 Python training tokens, and 2,097,152 development labels (4,096 windows) per domain. The *quick* development set is the first 262,144 of those labels. Reserved-test sets of 8,388,608 labels per domain were sealed and never used. The non-whitespace code CE used to measure adaptation excludes class W only.

3.5 Evaluation schedule and decision rules

Full development evaluations ran at 13 prefix points and at 25 continuation points per branch. Quick development evaluations ran at the switch and at 80 continuation points per branch, dense early on (Appendix A). Every full evaluation stores per-document and per-class sufficient statistics.

The secondary *matched-adaptation* analysis is fixed in advance. It fixes targets at the RMS Python branch’s quick-dev code CE at $s \in \{1525, 3050, 6104\}$. It finds the first downward crossing of each target by Taper-minus within an observed bracket of at most 100 updates (no extrapolation) and interpolates Taper-minus’s web CE at that crossing. It then compares the two models’ web forgetting relative to their own $s = 0$.

The decision rules in Table 1 are applied in order once all three seed groups are complete. They are screening criteria, not significance tests. With three seeds, the reported 95% interval $\bar{D} \pm 4.303 s_D / \sqrt{3}$ is descriptive.

3.6 Pre-specification and amendments

The hypothesis, design, estimands, evaluation schedule and decision thresholds come from a written protocol (“protocol v3”), drafted before the data were prepared. [Author: Confirm the protocol’s drafting date (the project export suggests 11–12 September 2026) and where it was kept.] The protocol was not lodged with an external registry, which is a limitation (Section 7). The amendments

Table 1: Pre-specified decision rules, applied in order. CE values are in nats/token; a bar denotes the mean over seeds.

#	Category	Condition
1	Invalid or incomplete	Missing seed group, unresolved correctness or data failure, nonfinite primary run, or resource termination
2	Comparability / adaptation limited	Any seed’s relative prefix web-CE gap $> 2\%$, or either condition’s Python branch improves non-W code CE by < 0.05
3	Opposite direction	$\bar{D} \leq -0.03$
4	Proceed to design next study	$\bar{D} \geq 0.03$; $D > 0$ in every seed; $\bar{D}(3050) \geq 0.015$; $\bar{Q} > 0.5 \bar{D}$; mean matched differential forgetting > 0 at the furthest common target
5	Transient only	$\bar{D} < 0.015$ and quick-dev $\bar{D}(s) \geq 0.05$ at some scheduled $s \leq 1000$
6	Stop: small observed effect	$\bar{D} < 0.015$
7	Inconclusive	Anything else

Table 2: Protocol amendments, all made before the primary run.

Date	Amendment
2026-09-28	NVIDIA T4 hardware: FP16 autocast with dynamic loss scaling replaces BF16; FP32 master weights, moments, normalization and EMA statistics, and loss. Results are not pooled with any BF16 run.
2026-09-30	Free cloud GPU time spread over sessions replaces the original 24-GPU-hour paid budget; the scientific allocation is unchanged.
2026-10-01	RMS and Taper-minus are trained as independent processes on two GPUs, with no shared state.
2026-10-03	Microbatch of 8 sequences $\times$ 4 accumulation steps (the protocol’s default layout; effective batch unchanged). For storage, model weights are kept only for the switch state and branch endpoints; the per-document and per-class statistics required at every full-development point are kept.

in Table 2 were made before any primary-run update; none of them changes the model, data, token budgets, seeds, endpoint or thresholds.

The primary run’s configuration (thresholds, schedule, batch layout and data hashes) was frozen into a hash (c18441799eee2b6a...) at launch, and the analysis is a deterministic function of the stored evaluation records. A public commit containing the protocol and the analysis code was made on 2026-10-03 at 16:07 UTC: after the primary run had started (2026-10-03 12:18 UTC) but before it finished (2026-10-03 21:40 UTC).

Before the primary run, engineering runs measured throughput and verified checkpoint resume. These included earlier preflights under a separate engineering seed, and a 25-minute smoke test that repeated roughly the first 2,300 prefix updates of seed 101 for both conditions. These runs were discarded. None of them reached the switch, so no continuation-phase quantity, and hence no value of D , was computed before the primary run. **[Author: Confirm that no earlier run (including exploratory runs on proxy corpora) produced a D -like contrast on these data.]**

4 Execution and implementation checks

The primary run took one 9.36-hour session on a cloud machine with two NVIDIA Tesla T4 GPUs (PyTorch 2.11.0, CUDA 12.8), with RMS on one GPU and Taper-minus on the other. All six condition-seed runs completed all 21,364 updates. There were no nonfinite losses, gradients or weights, and the loss scale never needed a retry (0 retries in total). Gradient clipping was active in 52–60% of updates for RMS and 61–70% for Taper-minus. GPU kernels used for the backward pass are nondeterministic, so results are reproducible to within numerical noise, not bitwise.

The training code is a single script that transcribes a separately tested reference implementation of the model, optimizer step and analysis. Before the run, it was checked against that reference in seven ways:

- (i) identical learning-rate and gate values at all 15,260 updates;
- (ii) identical paired initial tensors;
- (iii) FP32 training updates during the calibration phase that match the reference update to a relative tolerance of 2×10^{-5} ;
- (iv) bit-identical FP32 model outputs at updates spanning every TaperNorm gate state;
- (v) a vectorized evaluator that reproduces the reference evaluator’s CE and class sums on the real development data, to a relative tolerance of 10^{-9} ;
- (vi) identical decision categories on 3,000 randomly generated evidence sets;
- (vii) checkpoint resume whose deviation from an uninterrupted run is of the same order as run-to-run GPU nondeterminism.

The primary contrasts in Section 5 were also recomputed independently from the raw evaluation records, and they agree with the frozen analysis output.

5 Results

5.1 Validity checks

Table 3 shows that every guardrail passed.

- **Comparable prefixes.** The relative gap in prefix web CE was 0.28–0.34% (limit 2%), with Taper-minus slightly lower in every seed. Over the last five prefix evaluations, both conditions were still improving at nearly the same rate (mean slope -0.00244 vs. -0.00249 nats per million tokens).
- **Strong adaptation.** Both conditions adapted strongly to Python: non-W code CE fell by 3.34–3.60 nats (minimum 0.05), and the adaptation curves almost coincide (Figure 2c).
- **Large forgetting in both conditions.** The Python branch raised web CE by 1.63 to 1.69 nats/token relative to the switch, while web continuation lowered it by 0.12–0.13 (Figure 2a).

5.2 Primary endpoint

Table 4 gives the primary results. Mean $D = -0.0223$ nats/token (SD 0.0034; descriptive 95% interval $[-0.0308, -0.0139]$), and D is negative in all three seeds. Since $\bar{D} < 0.015$ and no early

Table 3: Validity checks and actual forgetting F (full-dev web CE, nats/token). “Gap” is the absolute prefix web-CE difference relative to RMS (limit 2%). “Code gain” is the Python branch’s reduction in non-W code CE from the switch to $s = 6,104$ (minimum 0.05).

Seed	Prefix web CE			F , web branch		F , Python branch		Code gain (R / T)
	RMS	Taper	Gap	RMS	Taper	RMS	Taper	
101	4.5846	4.5716	0.28%	−0.128	−0.128	+1.655	+1.636	3.55 / 3.55
102	4.5795	4.5666	0.28%	−0.123	−0.122	+1.688	+1.668	3.60 / 3.46
103	4.5931	4.5773	0.34%	−0.128	−0.124	+1.655	+1.633	3.35 / 3.34

Table 4: Primary contrast D at $s = 6,104$ and secondary contrasts (nats/token; full development set). Positive values mean more excess web forgetting for Taper-minus. “Matched” is the matched-adaptation differential forgetting at the furthest target reached by every seed ($s = 6,104$).

Seed	D	$D(3050)$	$D(1525)$	G_{web}	Q	Matched ($s=6104$)
101	−0.0200	+0.0109	+0.0369	−0.0006	−0.0194	−0.0308
102	−0.0207	−0.0124	+0.0224	−0.0009	−0.0199	−0.0182
103	−0.0262	+0.0193	−0.0286	−0.0042	−0.0220	−0.0286
Mean	−0.0223	+0.0059	+0.0102	−0.0019	−0.0204	−0.0259
SD	0.0034	0.0165	0.0344	0.0020	0.0014	0.0067

transient reached 0.05 (below), the pre-specified decision is **stop: small observed effect** (rule 6). The data provide no evidence for H1.

The other quantities point the same way as D :

- G_{web} is close to zero (mean −0.0019), so the difference sits in the Python branch itself: the direct differential forgetting Q averages −0.0204.
- The matched-adaptation contrast at $s = 6,104$ averages −0.0259, negative in every seed. On the quick development set, Taper-minus reached the RMS code-CE target before the corresponding RMS update in 9 of 9 seed–target pairs (Appendix B). At the furthest target, then, Taper-minus also showed slightly less web forgetting when compared at matched code adaptation.
- The per-document and per-class decompositions reconstruct D exactly. The effect is broad rather than driven by a few documents: median per-document D_i lies between −0.027 and −0.023. By token class, alphanumeric tokens contribute negatively in all 3 seeds and give the largest share in 2 of them; in seed 101, punctuation and symbol tokens contribute more (Appendix B).

5.3 Size and stability of the difference

The difference between the conditions is small in absolute terms: $|\bar{D}|$ is 1.3% of the mean Python-branch forgetting, which is 1.66 nats/token. Its sign was also not stable during continuation (Figure 2b):

- A single seed’s $D(s)$ changed by up to 0.045 between neighbouring full evaluations, and the largest $|D(s)|$ of any seed was 0.041.
- The seed-mean $D(s)$ ranged from −0.024 at $s = 5,795$ to +0.024 at $s = 2,745$.

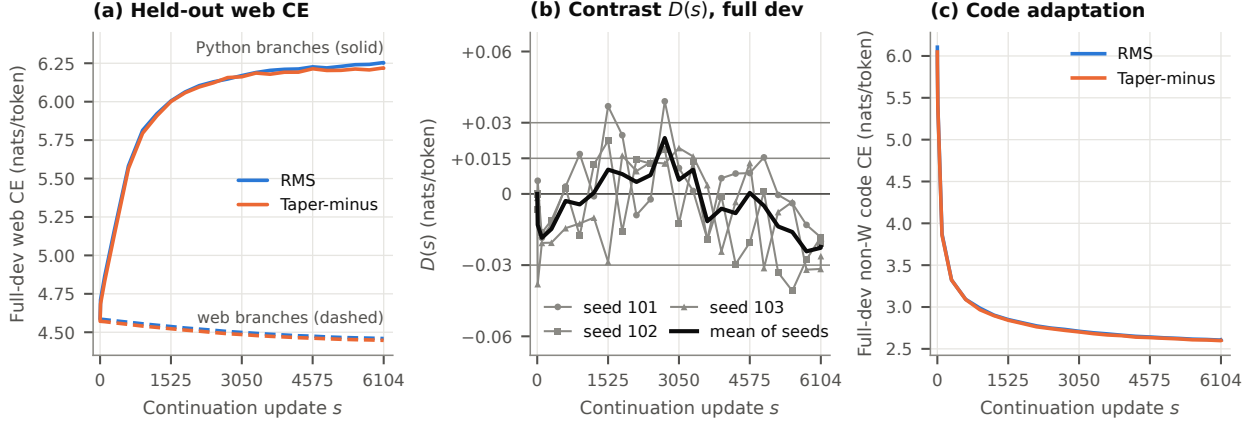


Figure 2: Continuation trajectories on the full development sets (means over three seeds unless stated). **(a)** Held-out web CE in all four branches. RMS and Taper-minus nearly coincide within each branch, and the Python branches show large forgetting. **(b)** The contrast $D(s)$ for each seed and their mean. The labelled horizontal lines are the pre-specified thresholds (+0.03 proceed, +0.015 small-effect bound, -0.03 opposite direction); the decision uses only $s = 6,104$ (and $s = 3,050$ for rule 4). **(c)** Non-whitespace code CE on the Python branch, showing near-identical adaptation in the two conditions.

- At $s = 1,525$ the seeds gave +0.037, +0.022 and -0.029; at $s = 3,050$ the mean was +0.0059, with mixed signs.

The quick-dev contrast never approached the transient threshold: its seed mean peaked at +0.015 (at $s = 400$) within the first 1,000 updates, against a threshold of 0.05.

The consistently negative endpoint is therefore one value from a fluctuating trajectory. It was pre-specified precisely so that no favourable checkpoint could be chosen afterwards.

6 Discussion

What the result shows. In this setting, replacing internal RMSNorm with TaperNorm did not increase persistent forgetting of web text after a switch to Python. The pre-specified screen for a meaningful excess ($\bar{D} \geq 0.015$) was not met, and every seed’s endpoint D was below zero. Under the protocol’s rules this direction is stopped. No seeds were added, no endpoint was changed and no follow-up was launched.

What it does not show. *Stop* is not evidence of equivalence (Lakens, 2017), and the negative endpoint is not evidence that TaperNorm protects against forgetting. That effect would have needed $\bar{D} \leq -0.03$, which was not reached. The intermediate-checkpoint fluctuations in Figure 2b are as large as the endpoint value, and three seeds give a fragile estimate of seed-to-seed variance (Dodge et al., 2020). The descriptive interval is conditional on one prepared corpus sample and one training recipe; it is not a population bound.

Why the concern may not have materialized (conjecture). We did not test mechanisms, but three features of the setting may matter.

- Taper-minus keeps the final RMSNorm, which Kanavalau et al. (2026) identify as the essential scale anchor.

- $\tilde{\gamma}$ and all weight matrices remain trainable after the taper, so scale changes can be absorbed cheaply.
- The forgetting here is very large in both conditions, which suggests that most of it comes from factors the two architectures share.

Each of these is a hypothesis for a future study, not a finding of this one.

7 Limitations

- **Scale.** The models have 17,718,784 parameters, about 73% of them in embeddings, and see 150M prefix tokens. Embedding drift may mediate or interact with the condition effect, and results may not transfer to larger models or longer training.
- **Breadth.** There is one domain pair (web to Python), one data sample, one optimizer recipe and one hardware and precision setting (FP16 on T4, not the protocol’s original BF16). The auxiliary-loss variant of TaperNorm (Taper-plus) was not run.
- **Statistics.** Three paired seeds share one prepared corpus, so uncertainty is conditional on that corpus, and the interval in Table 4 is descriptive. All measurements use development data; the reserved test sets remain sealed for any confirmatory study.
- **Pre-specification.** The protocol was kept privately and not registered externally. The public record of the analysis code postdates the start of the primary run (Section 3.6).
- **Data audit.** The near-duplicate search is approximate, and the random audit of 2,043 cross-split document pairs (0 found above threshold) has low power to detect rare leakage. Repository names are an incomplete proxy for forks.
- **Secondary analyses.** Matched adaptation interpolates between evaluated checkpoints and describes curves; it does not evaluate an actual model. Activation and attention diagnostics were logged but are not analysed here.

8 Conclusion

A small pilot, specified before training, found no evidence that TaperNorm increases persistent forgetting after a web-to-code domain shift: mean $D = -0.0223$ nats/token, against a pre-specified $+0.015$ bound. The difference pointed the other way at the endpoint, but it was small and unstable across checkpoints, so we draw no conclusion about protection. Testing H1 more broadly would need a new pre-specified study with larger models, more seeds and more than one domain shift. On its own, this pilot does not justify one.

Code and data availability

The code (data preparation, training, evaluation and the frozen analysis), the frozen configuration with data and source hashes, the decision report, and every evaluation record behind the tables and figures are available at [Author: repository URL, currently <https://github.com/Danieldotcomcoder/domain-shift-forgetting>]. The script `paper/make_assets.py` regenerates every number, table and figure in this paper from those records. C4 and The Stack are subject to their own licenses and terms of use; no corpus text is redistributed.

Acknowledgements

[**Author:** Disclose AI assistance as required by arXiv policy, for example: “The training and analysis code and a first draft of this manuscript were prepared with assistance from an AI system (Claude, Anthropic). The author designed the study, reviewed all code and text, and takes full responsibility for the content.” Adjust this to reflect your actual roles.]

References

- Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- Andrew Brock, Soham De, Samuel L. Smith, and Karen Simonyan. High-performance large-scale image recognition without normalization. In *Proceedings of the 38th International Conference on Machine Learning*, 2021.
- Andrei Z. Broder. On the resemblance and containment of documents. In *Proceedings of Compression and Complexity of SEQUENCES 1997*, pages 21–29, 1997.
- Jesse Dodge, Gabriel Ilharco, Roy Schwartz, Ali Farhadi, Hannaneh Hajishirzi, and Noah Smith. Fine-tuning pretrained language models: Weight initializations, data orders, and early stopping. *arXiv preprint arXiv:2002.06305*, 2020.
- Robert M. French. Catastrophic forgetting in connectionist networks. *Trends in Cognitive Sciences*, 3(4):128–135, 1999.
- Suchin Gururangan, Ana Marasović, Swabha Swayamdipta, Kyle Lo, Iz Beltagy, Doug Downey, and Noah A. Smith. Don’t stop pretraining: Adapt language models to domains and tasks. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8342–8360, 2020.
- Adam Ibrahim, Benjamin Thérien, Kshitij Gupta, Mats L. Richter, Quentin Anthony, Timothée Lesort, Eugene Belilovsky, and Irina Rish. Simple and scalable strategies to continually pre-train large language models. *Transactions on Machine Learning Research*, 2024.
- Andrei Kanavalau, Carmen Amo Alonso, and Sanjay Lall. Gated removal of normalization in transformers enables stable training and efficient inference. *arXiv preprint arXiv:2602.10408*, 2026.
- Andrej Karpathy. nanoGPT. <https://github.com/karpathy/nanoGPT>, 2022.
- James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.
- Denis Kocetkov, Raymond Li, Loubna Ben Allal, Jia Li, Chenghao Mou, Carlos Muñoz Ferrandis, Yacine Jernite, Margaret Mitchell, Sean Hughes, Thomas Wolf, Dzmitry Bahdanau, Leandro von Werra, and Harm de Vries. The stack: 3 TB of permissively licensed source code. *arXiv preprint arXiv:2211.15533*, 2022.

- Daniël Lakens. Equivalence tests: A practical primer for t tests, correlations, and meta-analyses. *Social Psychological and Personality Science*, 8(4):355–362, 2017.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
- Michael McCloskey and Neal J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. In *Psychology of Learning and Motivation*, volume 24, pages 109–165. Academic Press, 1989.
- Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. In *International Conference on Learning Representations*, 2018.
- Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018.
- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.
- Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. On layer normalization in the transformer architecture. In *Proceedings of the 37th International Conference on Machine Learning*, 2020.
- Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- Hongyi Zhang, Yann N. Dauphin, and Tengyu Ma. Fixup initialization: Residual learning without normalization. In *International Conference on Learning Representations*, 2019.

A Evaluation schedule

Full development evaluations (2,097,152 labels per domain). Prefix: global updates 763, 3052, 6104, 6409, 6714, 7019, 7324, 7629, 7934, 8239, 8544, 8849 and 9156. Continuation (each branch): $s \in \{0, 1, 10, 100\}$, every 305 updates from 305 to 6100, and 6104.

Quick development evaluations (262,144 labels per domain). At the switch, and at continuation $s \in \{0, 1, 2, 5, 10, 20\}$, every 50 updates from 50 to 1000, every 100 updates from 1100 to 6100, and $\{1525, 3050, 6104\}$ (80 points). Quick-dev values describe transients and feed matched adaptation. They never enter the full-dev primary contrast.

Diagnostics. Forward-only activation, gain and attention-entropy probes on the last 256 training windows of each domain, at the switch and at $s \in \{0, 10, 100, 1525, 6104\}$. They are logged but not analysed here.

B Additional results

Table 5: Matched-adaptation differential forgetting (nats/token) and the interpolated Taper-minus update at which it reached each RMS code-CE target, with the observed bracket in parentheses.

Seed	Matched differential forgetting			Taper update at match (bracket)		
	$s=1525$	$s=3050$	$s=6104$	$s=1525$	$s=3050$	$s=6104$
101	−0.0075	−0.0038	−0.0308	1447 (1400–1500)	2740 (2700–2800)	5198 (5100–5200)
102	+0.0094	−0.0106	−0.0182	1475 (1400–1500)	2776 (2700–2800)	5775 (5700–5800)
103	−0.0259	+0.0145	−0.0286	1519 (1500–1525)	3024 (3000–3050)	5862 (5800–5900)

Table 6: Decomposition of the endpoint D (nats/token) into disjoint token classes (each class’s contribution weighted by its label share; the four contributions sum to D), and summaries of per-document effects D_i over the 4,409 web development documents. “Top-1% sum” is the signed contribution of the 1% of documents with the largest positive contributions.

Seed	Contribution to D by token class				Per-document D_i		
	W	A	P	X	Median	1%-trimmed	Top-1% sum
101	+0.0038	−0.0098	−0.0121	−0.0019	−0.0232	−0.0203	+0.0018
102	−0.0019	−0.0155	+0.0014	−0.0047	−0.0257	−0.0210	+0.0020
103	+0.0008	−0.0227	−0.0055	+0.0012	−0.0272	−0.0265	+0.0020
Labels	33,807	1,786,076	230,998	46,271	4,409 documents		